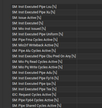

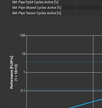

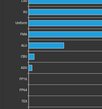

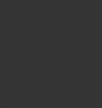

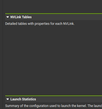

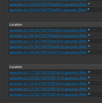

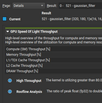

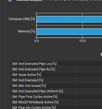

Page: Details
Result: 0 - 521 - gaussian_filter
Add Baseline
Apply Rules
Occupancy Calculator
Save as PDF

	Result	Time	Cycles	Regs	GPU	SM Frequency	CC	Process
Current	521 - gaussian_filter (320, 180, 1)x(16, 16, 1)	12.54 msecond	12,690,839	31	0 - NVIDIA GeForce RTX 2060	1.01 cycle/nsecond	7.5	[31744] project_canny

GPU Speed Of Light Throughput

High-level overview of the throughput for compute and memory resources of the GPU. For each unit, the throughput reports the achieved percentage of utilization with respect to the theoretical maximum. Breakdowns show the throughput for each individual sub-metric of Compute and Memory to clearly identify the highest contributor. High-level overview of the utilization for compute and memory resources of the GPU presented as a roofline chart.

Compute (SM) Throughput [%]	91.48	Duration [msecond]	12.54
Memory Throughput [%]	91.48	Elapsed Cycles [cycle]	12,690,839
L1/TEX Cache Throughput [%]	93.50	SM Active Cycles [cycle]	12,642,879.80
L2 Cache Throughput [%]	1.69	SM Frequency [cycle/nsecond]	1.01
DRAM Throughput [%]	0.85	DRAM Frequency [cycle/nsecond]	5.49

High Throughput The kernel is utilizing greater than 80.0% of the available compute or memory performance of the device. To further improve performance, work will likely need to be shifted from the most utilized to another unit. Start by analyzing workloads in the ▷ Compute Workload Analysis section.

Roofline Analysis The ratio of peak float (fp32) to double (fp64) performance on this device is 32:1. The kernel achieved 22% of this device's fp32 peak performance and 0% of its fp64 peak performance. See the ⊕ Kernel Profiling Guide for more details on roofline analysis.

GPU Throughput

Compute (SM) [%]
Memory [%]
0.0
10.0
20.0
30.0
40.0
50.0
60.0
70.0
80.0
90.0
100.0
Speed Of Light (SOL) [%]

Compute Throughput Breakdown

SM: Inst Executed Pipe Lsu [%]	91.48
SM: Inst Executed Pipe Xu [%]	87.90
SM: Issue Active [%]	55.07
SM: Inst Executed [%]	51.97
SM: Mio Inst Issued [%]	30.74
SM: Inst Executed Pipe Uniform [%]	23.58
SM: Pipe Fma Cycles Active [%]	23.05
SM: Mio2rf Writeback Active [%]	22.56
SM: Pipe Alu Cycles Active [%]	7.55
SM: Inst Executed Pipe Cbu Pred On Any [%]	4.62
SM: Mio Pq Read Cycles Active [%]	0.95
SM: Mio Pq Write Cycles Active [%]	0.90
SM: Inst Executed Pipe Adu [%]	0.73
SM: Inst Executed Pipe Fp16 [%]	0
SM: Inst Executed Pipe Ipa [%]	0
SM: Inst Executed Pipe Tex [%]	0
IDC: Request Cycles Active [%]	0
SM: Pipe Fp64 Cycles Active [%]	0
SM: Pipe Shared Cycles Active [%]	0
SM: Pipe Tensor Cycles Active [%]	0

Memory Throughput Breakdown

L1: Lsuin Requests [%]	91.48
L1: Data Pipe Lsu Wavefronts [%]	46.75
L1: Lsu Writeback Active [%]	45.20
L1: Data Bank Reads [%]	6.68
L2: Xbar2lts Cycles Active [%]	1.69
L2: T Sectors [%]	1.66
L2: T Tag Requests [%]	1.41
L2: Lts2xbar Cycles Active [%]	1.33
L1: M L1tex2xbar Req Cycles Active [%]	1.26
L1: M Xbar2l1tex Read Sectors [%]	1.01
DRAM: Cycles Active [%]	0.85
DRAM: Dram Sectors [%]	0.61
L2: D Sectors [%]	0.49
L1: Data Bank Writes [%]	0.22
L2: D Sectors Fill Device [%]	0.16
L1: Texin Sm2tex Req Cycles Active [%]	0.02
L1: F Wavefronts [%]	0.00
L2: D Sectors Fill Sysmem [%]	0.00
L1: Tex Writeback Active [%]	0
L2: D Atomic Input Cycles Active [%]	0
L1: Data Pipe Tex Wavefronts [%]	0

Floating Point Operations Roofline

Performance [FLOP/s] (1 = 1E+12)
100
10
1
0.1
0.1
1
10
100
1,000
10,000
100,000
Arithmetic Intensity [FLOP/byte]

Compute Workload Analysis

Detailed analysis of the compute resources of the streaming multiprocessors (SM), including the achieved instructions per clock (IPC) and the utilization of each available pipeline. Pipelines with very high utilization might limit the overall performance.

Executed Ipc Elapsed [inst/cycle]	2.08	SM Busy [%]	87.96
Executed Ipc Active [inst/cycle]	2.08	Issue Slots Busy [%]	55.10
Issued Ipc Active [inst/cycle]	2.20		

Balanced FMA is the highest-utilized pipeline (23.1%) based on active cycles, taking into account the rates of its different instructions. It executes 32-bit floating point (FADD, FMUL, FMAD, ...) and integer (IMUL, IMAD) operations. It is well-utilized, but should not be a bottleneck.

Pipe Utilization (% of active cycles)

FMA
ALU
FP64
Shared (FP16+Tensor)
Tensor (All)
Tensor (FP)
Tensor (INT)
0.0
25.0
50.0
75.0
100.0
Utilization [%]

Pipe Utilization (% of peak instructions executed)

LSU
XU
Uniform
FMA
ALU
CBU
ADU
FP16
FP64
TEX
Tensor (FP)
Tensor (INT)
0.0
25.0
50.0
75.0
100.0
Utilization [%]

Memory Workload Analysis

Detailed analysis of the memory resources of the GPU. Memory can become a limiting factor for the overall kernel performance when fully utilizing the involved hardware units (Mem Busy), exhausting the available communication bandwidth between those units (Max Bandwidth), or by reaching the maximum throughput of issuing memory instructions (Mem Pipes Busy). Detailed chart of the memory units. Detailed tables with data for each memory unit.

Memory Throughput [Gbyte/second]	2.25	Mem Busy [%]	46.75
L1/TEX Hit Rate [%]	45.54	Max Bandwidth [%]	91.48
L2 Hit Rate [%]	94.32	Mem Pipes Busy [%]	91.48

L1TEX Global Load Access Pattern The memory access pattern for global loads in L1TEX might not be optimal. On average, this kernel accesses 1.0 bytes per thread per memory request; but the address pattern, possibly caused by the stride between threads, results in 2.6 sectors per request, or 2.6*32 = 84.8 bytes of cache data transfers per request. The optimal thread address pattern for 1.0 byte accesses would result in 1.0*32 = 32.0 bytes of cache data transfers per request, to maximize L1TEX cache performance. Check the ▷ Source Counters section for uncoalesced global loads.

L1TEX Global Store Access Pattern The memory access pattern for global stores in L1TEX might not be optimal. On average, this kernel accesses 1.0 bytes per thread per memory request; but the address pattern, possibly caused by the stride between threads, results in 2.0 sectors per request, or 2.0*32 = 64.0 bytes of cache data transfers per request. The optimal thread address pattern for 1.0 byte accesses would result in 1.0*32 = 32.0 bytes of cache data transfers per request, to maximize L1TEX cache performance. Check the ▷ Source Counters section for uncoalesced global stores.

L2 Load Access Pattern The memory access pattern for loads from L1TEX to L2 is not optimal. The granularity of an L1TEX request to L2 is a 128 byte cache line. That is 4 consecutive 32-byte sectors per L2 request. However, this kernel only accesses an average of 1.2 sectors out of the possible 4 sectors per cache line. Check the ▷ Source Counters section for uncoalesced loads and try to minimize how many cache lines need to be accessed per memory request.

L2 Store Access Pattern The memory access pattern for stores from L1TEX to L2 is not optimal. The granularity of an L1TEX request to L2 is a 128 byte cache line. That is 4 consecutive 32-byte sectors per L2 request. However, this kernel only accesses an average of 1.0 sectors out of the possible 4 sectors per cache line. Check the ▷ Source Counters section for uncoalesced stores and try to minimize how many cache lines need to be accessed per memory request.

Memory Chart

Show As: Transfer Size

Kernel
3.39 M Inst
Global
2.93 M Req
460.80 K Req
0.00 Inst
Local
0.00 Req
0.00 Req
0.00 Inst
Texture
0.00 Req
0.00 Inst
Surface
0.00 Req
0.00 Req
169.29 M Inst
Shared
166.35 M Req
2.94 M Req
L1/TEX Cache
Hit Rate: 45.54 %
Shared Memory
123.00 MB
29.49 MB
L2 Cache
Hit Rate: 94.32 %
0.00 B
0.00 B
System Memory
14.75 MB
13.43 MB
Device Memory
0.00 B
0.00 B
Peer Memory
% Peak
100%
80%
60%
40%
20%
0%

Shared Memory

	Instructions	Requests	Wavefronts	% Peak	Bank Conflicts
Shared Load	166,348,800	166,348,800	166,348,800	43.83	0
Shared Load Matrix	0	0			
Shared Store	2,937,600	2,937,600	2,937,681	0.77	0
Shared Atomic	0	0	0	0	0
Other	-	-	979,198	0.26	0
Total	169,286,400	169,286,400	170,265,679	44.86	0

L1/TEX Cache

	Instructions	Requests	Wavefronts	% Peak	Sectors	Sectors/Req	Hit Rate	Bytes	Sector Misses to L2	% Peak to L2	Returns to SM	% Peak to SM
Local Load	0	0	0	0	0	0	0	0	3,846,150	1.01	4,283,075	1.13
Global Load	2,926,868	2,926,868	4,074,006	1.07	7,753,863	2.65	50.94	248,123,616				
Surface Load	0	0	0	0	0	0	0	0	0	0	0	0
Texture Load	0	0	0	0	0	0	0	0	0	0		
Global Store	460,800	460,800	657,408	0.17	921,600	2	0.10	29,491,200	921,600	0.24	-	-
Local Store	0	0	0	0	0	0	0	0			-	-
Surface Store	0	0	0	0	0	0	0	0	0	0	-	-
Global Reduction	0	0	0	0	0	0	0	0	0	0	-	-
Surface Reduction	0	0	0	0	0	0	0	0	0	0	-	-
Global Atomic ALU	0	0	0	0	0	0	0	0	0	0	see above	see above
Global Atomic CAS	0											
Surface Atomic ALU	0	0	0	0	0	0	0	0	0	0	see above	see above
Surface Atomic CAS	0											
Loads	2,926,868	2,926,868	4,074,006	1.07	7,753,863	2.65	50.94	248,123,616	3,846,150	1.01	4,283,075	1.13
Stores	460,800	460,800	657,408	0.17	921,600	2	0.10	29,491,200	921,600	0.24	-	-
Atomics & Reductions	0	0	0	0	0	0	0	0	0	0	-	-
Total	3,387,668	3,387,668	4,731,414	1.25	8,675,463	2.56	45.54	277,614,816	4,767,750	1.26	4,283,075	1.13

L2 Cache

	Requests	Sectors	Sectors/Req	% Peak	Hit Rate	Bytes	Throughput	Sector Misses to Device	Sector Misses to System	Sector Misses to Peer
L1/TEX Load	3,126,056	3,842,225	1.23	1.33	88.58	122,951,200	9,801,769,419.35	460,800	0	0
L1/TEX Store	921,598	921,600	1.00	0.32	100	29,491,200	2,351,062,391.42	0	0	0
L1/TEX Atomic ALU	0	0	0	0	0	0	0	0	0	0
L1/TEX Atomic CAS	0	0	0	0	0	0	0	0	0	0
L1/TEX Reduction	0	0	0	0	0	0	0	0	0	0
L1/TEX Total	4,049,037	4,764,054	1.18	1.65	90.71	152,449,728	12,153,416,004.88	460,800	0	0
ECC Total	-	0	-	0	-	0	0	0	-	-
GPU Total	4,057,256	4,776,896	1.18	1.66	90.58	152,860,672	12,186,176,793.97	469,811	4	0

Device Memory

	Sectors	% Peak	Bytes	Throughput
Load	460,907	0.45	14,749,024	1,175,804,159.77
Store	419,679	0.41	13,429,728	1,070,628,812.25
Total	880,586	0.85	28,178,752	2,246,432,972.02

Scheduler Statistics

Summary of the activity of the schedulers issuing instructions. Each scheduler maintains a pool of warps that it can issue instructions for. The upper bound of warps in the pool (Theoretical Warps) is limited by the launch configuration. On every cycle each scheduler checks the state of the allocated warps in the pool (Active Warps). Active warps that are not stalled (Eligible Warps) are ready to issue their next instruction. From the set of eligible warps the scheduler selects a single warp from which to issue one or more instructions (Issued Warp). On cycles with no eligible warps, the issue slot is skipped and no instruction is issued. Having many skipped issue slots indicates poor latency hiding.

Active Warps Per Scheduler [warp]	7.58	No Eligible [%]	44.90
Eligible Warps Per Scheduler [warp]	1.45	One or More Eligible [%]	55.10
Issued Warp Per Scheduler	0.55		

Issue Slot Utilization Every scheduler is capable of issuing one instruction per cycle, but for this kernel each scheduler only issues an instruction every 1.8 cycles. This might leave hardware resources underutilized and may lead to less optimal performance. Out of the maximum of 8 warps per scheduler, this kernel allocates an average of 7.58 active warps per scheduler, but only an average of 1.45 warps were eligible per cycle. Eligible warps are the subset of active warps that are ready to issue their next instruction. Every cycle with no eligible warp results in no instruction being issued and the issue slot remains unused. To increase the number of eligible warps, avoid possible load imbalances due to highly different execution durations per warp. Reducing stalls indicated on the ▷ Warp State Statistics and ▷ Source Counters sections can help, too.

Warps Per Scheduler

GPU Maximum Warps Per Scheduler
Theoretical Warps Per Scheduler
Active Warps Per Scheduler
Eligible Warps Per Scheduler
Issued Warp Per Scheduler
0.0
4.0
8.0
12.0
16.0

Warp State Statistics

Analysis of the states in which all warps spent cycles during the kernel execution. The warp states describe a warp's readiness or inability to issue its next instruction. The warp cycles per instruction define the latency between two consecutive instructions. The higher the value, the more warp parallelism is required to hide this latency. For each warp state, the chart shows the average number of cycles spent in that state per issued instruction. Stalls are not always impacting the overall performance nor are they completely avoidable. Only focus on stall reasons if the schedulers fail to issue every cycle. When executing a kernel with mixed library and user code, these metrics show the combined values.

Warp Cycles Per Issued Instruction [cycle]	13.76	Avg. Active Threads Per Warp	31.58
Warp Cycles Per Executed Instruction [cycle]	14.58	Avg. Not Predicated Off Threads Per Warp	31.29

mio_throttle On average, each warp of this kernel spends 6.1 cycles being stalled waiting for an MIO instruction queue to be not full. This represents about 44.1% of the total average of 13.8 cycles between issuing two instructions. This stall reason is high in cases of utilization of the MIO pipelines, which include special math instructions, dynamic branches, as well as shared memory instructions. When caused by shared memory accesses, trying to use fewer but wider loads can reduce pipeline pressure.

Warp Stall Check the ▷ Source Counters section for the top stall locations in your source based on sampling data. The ⊕ Kernel Profiling Guide provides more details on each stall reason.

Warp State (All Cycles)

Warp States
Stall MIO Throttle
Stall IMC Miss
Stall Not Selected
Stall Wait
Selected
Stall Barrier
Stall Long Scoreboard
Stall No Instruction
Stall Short Scoreboard
Stall Branch Resolving
Stall Math Pipe Throttle
Stall Drain
Stall Dispatch Stall
Stall Misc
Stall LG Throttle
Stall Membar
Stall Sleeping
Stall Tex Throttle
0.0
2.0
4.0
6.0
8.0
Cycles per Instruction

Instruction Statistics

Statistics of the executed low-level assembly instructions (SASS). The instruction mix provides insight into the types and frequency of the executed instructions. A narrow mix of instruction types implies a dependency on few instruction pipelines, while others remain unused. Using multiple pipelines allows hiding latencies and enables parallel execution. Note that 'Instructions/Opcode' and 'Executed Instructions' are measured differently and can diverge if cycles are spent in system calls.

Executed Instructions [inst]	789,048,954	Avg. Executed Instructions Per Scheduler [inst]	6,575,407.95
Issued Instructions [inst]	836,003,473	Avg. Issued Instructions Per Scheduler [inst]	6,966,695.61

Executed Instruction Mix

Opcodes
ULDC
LDS
I2F
FFMA
ISETP
IADD3
BRA
UIADD3
IMAD
BSYNC
BSSY
BMOV
MOV
STS
LDG
S2R
SHF
LEA
EXIT
UMOV
STG
LOP3
IMNMX
F2I
BAR
0.0
50,000,000.0
100,000,000.0
150,000,000.0
200,000,000.0
Executed Instructions/Opcode

NVLink Topology

NVLink Topology diagram shows logical NVLink connections with transmit/receive throughput.

NVLink Topology

The system does not have any NVLink connections.

NVLink Tables

Detailed tables with properties for each NVLink.

Logical NVLink Properties

The system does not have any NVLink connections.

Launch Statistics

Summary of the configuration used to launch the kernel. The launch configuration defines the size of the kernel grid, the division of the grid into blocks, and the GPU resources needed to execute the kernel. Choosing an efficient launch configuration maximizes device utilization.

Grid Size	57,600	Function Cache Configuration	CachePreferNone
Registers Per Thread [register/thread]	31	Static Shared Memory Per Block [Kbyte/block]	1.16
Block Size	256	Dynamic Shared Memory Per Block [byte/block]	0
Threads [thread]	14,745,600	Driver Shared Memory Per Block [byte/block]	0
Waves Per SM	480	Shared Memory Configuration Size [Kbyte]	32.77

Occupancy

Occupancy is the ratio of the number of active warps per multiprocessor to the maximum number of possible active warps. Another way to view occupancy is the percentage of the hardware's ability to process warps that is actively in use. Higher occupancy does not always result in higher performance, however, low occupancy always reduces the ability to hide latencies, resulting in overall performance degradation. Large discrepancies between the theoretical and the achieved occupancy during execution typically indicates highly imbalanced workloads.

Theoretical Occupancy [%]	100	Block Limit Registers [block]	8
Theoretical Active Warps per SM [warp]	32	Block Limit Shared Mem [block]	25
Achieved Occupancy [%]	94.85	Block Limit Warps [block]	4
Achieved Active Warps Per SM [warp]	30.35	Block Limit SM [block]	16

Occupancy Limiters This kernel's theoretical occupancy is not impacted by any block limit.

Impact of Varying Register Count Per Thread

Warp Occupancy
64
48
32
16
0
0
8
16
24
32
40
48
56
64
72
80
88
96
104
112
120
128
136
144
152
160
168
176
184
192
200
208
216
224
232
240
248
256
Registers Per Thread

Impact of Varying Block Size

Warp Occupancy
48
36
24
12
0
0
32
64
96
128
160
192
224
256
288
320
352
384
416
448
480
512
544
576
608
640
672
704
736
768
800
832
864
896
928
960
992
1,024
Block Size

Impact of Varying Shared Memory Usage Per Block

Warp Occupancy
64
48
32
16
0
0
2,032
4,064
6,096
8,128
10,160
12,192
14,224
16,256
18,288
20,320
22,352
24,384
26,416
28,448
30,480
32,512
34,544
36,576
38,608
40,640
42,672
44,704
46,736
48,768
50,800
52,832
54,864
56,896
58,928
60,960
62,992
65,024
Shared Memory Per Block

Source Counters

Source metrics, including branch efficiency and sampled warp stall reasons. Warp Stall Sampling metrics are periodically sampled over the kernel runtime. They indicate when warps were stalled and couldn't be scheduled. See the documentation for a description of all stall reasons. Only focus on stalls if the schedulers fail to issue every cycle.

Branch Instructions [inst]	25,294,418	Branch Efficiency [%]	94.52
Branch Instructions Ratio [%]	0.03	Avg. Divergent Branches	8,226.82

Uncoalesced Global Accesses This kernel has uncoalesced global accesses resulting in a total of 5326186 excessive sectors (61% of the total 8713854 sectors). Check the L2 Theoretical Sectors Global Excessive table for the primary source locations. The ⊕ CUDA Programming Guide had additional information on reducing uncoalesced device memory accesses.

L2 Theoretical Sectors Global Excessive

Location	Value	Value (%)
gaussian.cu:106 (0x77d47f25a750 in gaussian_filter) ↗	1,946,218	37
gaussian.cu:106 (0x77d47f25a680 in gaussian_filter) ↗	1,946,218	37
gaussian.cu:106 (0x77d47f25a820 in gaussian_filter) ↗	972,950	18
gaussian.cu:127 (0x77d47f25b1d0 in gaussian_filter) ↗	460,800	9

Warp Stall Sampling (All Cycles)

Location	Value	Value (%)
gaussian.cu:123 (0x77d47f25acb0 in gaussian_filter) ↗	16,299	4
gaussian.cu:117 (0x77d47f25ac30 in gaussian_filter) ↗	12,474	3
gaussian.cu:106 (0x77d47f25a6a0 in gaussian_filter) ↗	12,351	3
gaussian.cu:123 (0x77d47f25ad60 in gaussian_filter) ↗	10,537	3
gaussian.cu:123 (0x77d47f25adf0 in gaussian_filter) ↗	9,134	2

Warp Stall Sampling (Not-issued Cycles)

Location	Value	Value (%)
gaussian.cu:123 (0x77d47f25acb0 in gaussian_filter) ↗	6,955	4
gaussian.cu:106 (0x77d47f25a6a0 in gaussian_filter) ↗	5,939	4
gaussian.cu:117 (0x77d47f25ac30 in gaussian_filter) ↗	5,903	4
gaussian.cu:123 (0x77d47f25ad60 in gaussian_filter) ↗	4,518	3
gaussian.cu:106 (0x77d47f25a770 in gaussian_filter) ↗	4,274	3

Most Instructions Executed

Location	Value	Value (%)
gaussian.cu:117 (0x77d47f25b180 in gaussian_filter) ↗	8,755,200	1
gaussian.cu:123 (0x77d47f25b170 in gaussian_filter) ↗	8,755,200	1
gaussian.cu:123 (0x77d47f25b160 in gaussian_filter) ↗	8,755,200	1
gaussian.cu:117 (0x77d47f25b150 in gaussian_filter) ↗	8,755,200	1
gaussian.cu:123 (0x77d47f25b140 in gaussian_filter) ↗	8,755,200	1